SituationReport

testdata_generator

Manuel d'utilisation

Espaces de données démo et données Jira synthétiques pour SituationReport

---

situation-report -- github.com/Jaegerfeld/situation-report

Pour les Agile Coaches, PI Managers et testeurs bêta -- Version 0.54.0

1 Qu'est-ce que le Générateur de données de test ?

Le **Générateur de données de test** produit des données inventées mais réalistes pour SituationReport — pour démontrer, apprendre et tester le rapport de portefeuille, le briefing delta, le dossier de conférence et les registres sans toucher à de vraies données de projet. Aucune personne n'apparaît ; la même graine donne le même espace de données.

Le générateur a deux voies. L'**espace de données** est la voie par défaut de la bêta : en trois étapes naît un dossier de portefeuille complet avec structure, histoire, exports Jira par ART, registres, données brutes des fournisseurs, une série d'états et les briefings delta déjà rendus. L'**ART seul** est la voie classique : un fichier d'export Jira à partir d'un fichier de workflow, avec tous les réglages de cycle time et de motifs de flux (chapitre 8).

Quelle voie, quand ?
- **Espace de données (fenêtre ou navigateur) :** formation, présentation, tutoriels, tests de la chaîne de portefeuille — tout ce qui demande une image de situation entière.
- **ART seul :** un export Jira pour transform_data et build_reports, des motifs de flux ciblés pour la formation aux nuages de points de cycle time, la poursuite d'un extrait antérieur.

Ce manuel décrit d'abord l'espace de données (chapitres 2 à 7), puis le générateur classique d'ART seul (chapitre 8). Qui ne cherche que la minute jusqu'au premier rapport de portefeuille prend le tutoriel « Espace de données démo en une minute » (docs/tutorials/demo-data-room.md, aussi en PDF).

Usages typiques :
- **Tester une nouvelle installation :** la chaîne tourne-t-elle des données brutes au rapport de portefeuille ?
- **Démos et présentations :** une situation avec valeur aberrante, source faible et coutures, sans question de confidentialité
- **Formation :** la même histoire pour tous les participants, reproductible par la graine
- **Développement :** test de charge avec une grande structure (échelle L, jusqu'à 49 ARTs)

État : bêta, octobre 2026. L'interface de l'espace de données est nouvelle ; l'onglet des profils par ART dans l'interface manque encore (ligne de commande : --art-profiles).

2 Démarrage

2.1 Paquet portable (recommandé)

Le paquet portable embarque Python — rien à installer. Après décompression, quatre fichiers de démarrage du générateur sont prêts :

Fichier de démarrage (Windows)	Ce qu'il ouvre
`DatenraumFenster.bat`	**Espace de données en fenêtre** — la voie par défaut. Trois étapes, aucun port réseau, fonctionne aussi sur les ordinateurs d'entreprise.
`SituationReport.bat`	Le lanceur avec tous les modules ; il porte les cartes « Espace de données (fenêtre) » et « Espace de données (navigateur) » ainsi que « Testdata Generator » pour l'ART seul.
`DatenraumBrowser.bat`	**Espace de données dans le navigateur** — les mêmes trois étapes en page web. Démarre un serveur local sur 127.0.0.1 avec une clé aléatoire dans le lien. Ordinateurs privés seulement : le logiciel de sécurité des ordinateurs d'entreprise bloque et signale les serveurs locaux.
`TestdataGenerator.bat`	**ART seul** — la fenêtre classique avec fichier de workflow, réglages et motifs de flux (chapitre 8).

macOS : les mêmes noms en .command (clic droit → Ouvrir). Linux : en .sh (python3 et python3-tk requis). Le premier démarrage installe une fois un environnement Python — il faut Internet pour cela.

Note pour Windows : si le démarrage signale que Python est introuvable, Windows avait bloqué le fichier ZIP. Clic droit sur le ZIP → Propriétés → Sécurité → Débloquer → OK → décompresser à nouveau.

2.2 Depuis le code source (Python requis)

Les trois interfaces correspondent à trois appels : fenêtre, navigateur, ART seul.

```
python -m testdata_generator --dataspace
python -m testdata_generator --web
python -m testdata_generator
```

2.3 Fenêtre ou navigateur ?

Les deux montrent les mêmes trois étapes avec les mêmes réglages et produisent le même espace de données. La fenêtre est une fenêtre de programme ordinaire (tkinter) et n'a besoin de rien d'autre. La page du navigateur est un peu plus agréable à lire mais passe par un serveur local — sur un ordinateur d'entreprise, la sécurité des postes l'a bloqué et signalé au premier test. D'où la règle : la fenêtre d'abord, le navigateur seulement en privé.

2.4 Langues

Interface, histoire, README, progression, journal et les briefings delta générés existent en allemand, anglais, roumain, portugais et français. Le générateur démarre dans la langue réglée dans le lanceur ; le sélecteur en haut à droite de la fenêtre et de la page la change — les saisies sont conservées. Une exécution a une langue : interface et textes générés concordent. Sans réglage du lanceur et sans `--lang`, la langue est l'anglais. Le roumain, le portugais et le français sont des ébauches encore à relire par des locuteurs natifs.

3 L'espace de données en trois étapes

Fenêtre et page du navigateur guident à travers trois étapes : **Scénario** (quelle situation), **Périmètre & sources** (quelle taille, quelle durée, quel réalisme) et **Générer** (où, et c'est parti). Les valeurs par défaut produisent la situation démo qu'utilisent aussi les tutoriels et manuels des modules de portefeuille.

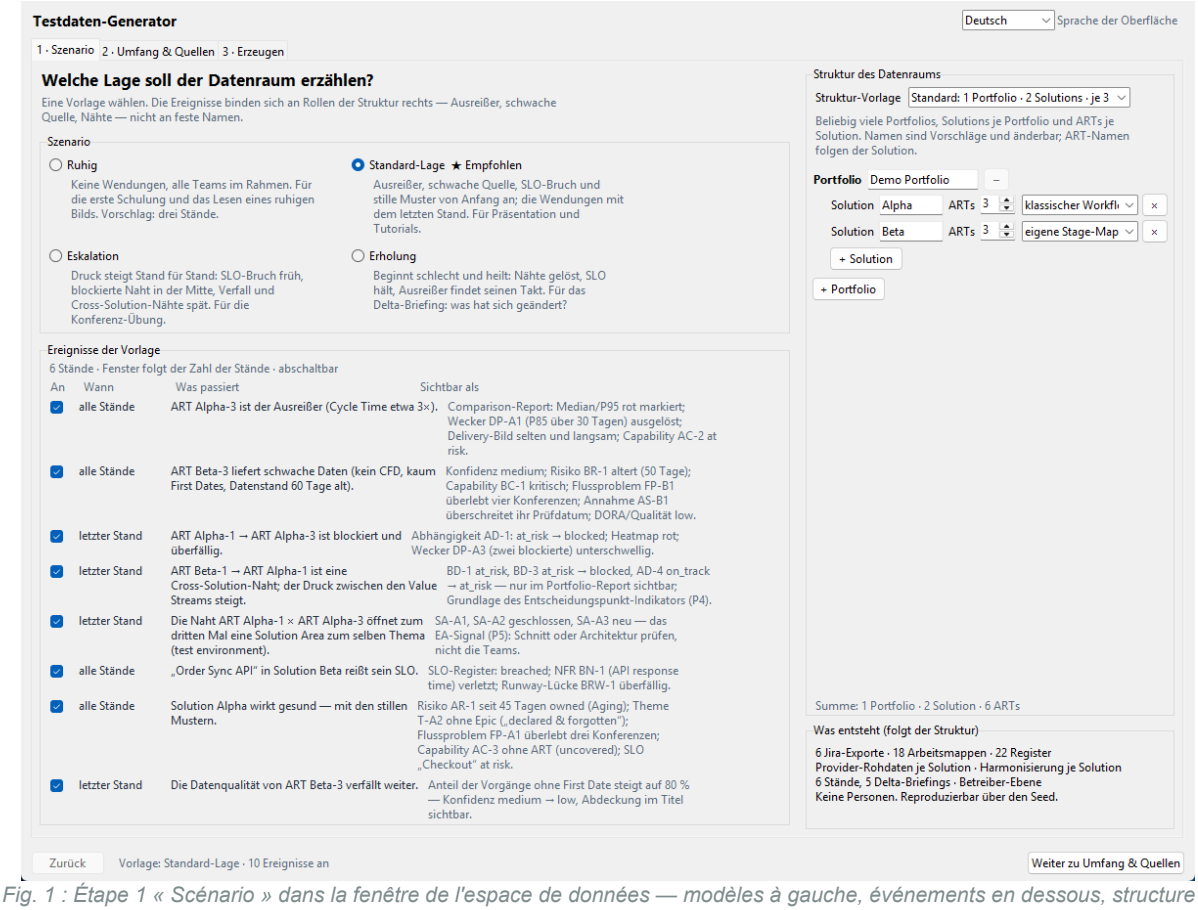


Fig. 1 : Étape 1 « Scénario » dans la fenêtre de l'espace de données — modèles à gauche, événements en dessous, structure à droite (interface allemande)

3.1 Étape 1 — Scénario

À gauche, vous choisissez un **modèle** d'histoire. Les événements se lient aux rôles de la structure à droite — valeur aberrante, source faible, coutures — et non à des noms fixes.

Modèle	Ce qu'il raconte	États
Calme	Aucun retournement, toutes les équipes dans le cadre. Pour la première formation et la lecture d'une image tranquille. Qui donne l'alarme ici a mal lu l'image.	3
Situation standard (recommandé)	Valeur aberrante, source faible, rupture de SLO et motifs silencieux dès le début ; les retournements arrivent avec le dernier état. Pour présentations et tutoriels.	6

Modèle	Ce qu'il raconte	États
Escalade	La pression monte état après état : rupture de SLO tôt, couture bloquée au milieu, dégradation et coutures inter-solutions tard. Pour l'exercice de conférence.	6
Rétablissement	Commence mal et guérit : coutures résolues, le SLO tient, la valeur aberrante trouve son rythme. Pour le briefing delta : qu'est-ce qui a changé ?	6

Événements du modèle. Le tableau en dessous indique pour chaque événement quand il s'applique (tous les états, à partir de l'état n, dernier état), ce qui se passe et sous quelle forme il devient visible dans le rapport. Chaque événement a une case à cocher ; les événements désactivés sont absents de l'espace de données. Un événement dont le prérequis est désactivé reste désactivé, avec une note. La fenêtre d'un événement suit le nombre d'états de l'étape 2. Le chapitre 5 explique les dix événements.

Structure de l'espace de données (droite). Un modèle de structure — Petite (1 portefeuille · 1 solution · 2 ARTs), Standard (1 · 2 · 3 chacune), Grande (5 portefeuilles · 3–4 solutions · 1–6 ARTs) — ou la vôtre : ajouter ou retirer des portefeuilles et des solutions, compter les ARTs par solution, changer les noms (les noms d'ART suivent la solution). Par solution, vous choisissez le workflow classique ou une stage map propre. Au moins un ART par solution.

Ce qui est produit. L'encadré en bas à droite fait le compte : exports Jira, classeurs, registres, états et briefings delta pour la structure choisie (standard : 6 exports, 18 classeurs, 22 registres, 6 états, 5 deltas). À partir de 30 ARTs il avertit : la génération prend alors nettement plus de temps.

3.2 Étape 2 — Périmètre & sources

Les valeurs par défaut suffisent pour la formation et la présentation.

Réglage	Défaut	Signification
États	6	2 à 12 états espacés de 14 jours ; chaque état est un instantané figé, chaque paire consécutive un briefing delta prêt. Le modèle « Calme » en propose 3.
Périmètre	M	S = ancres de l'histoire seulement, registres courts ; M = registres réalistes ; L = test de charge avec registres pleins (heatmap dense, matrice de feuille de route complète). Les ancres et leurs histoires sont identiques à chaque niveau.
Réalisme	60	Désordre des parcours de statut : sauts, abandons, mises en attente, boucles, réouvertures. 0 = lisse, 100 = comme en vrai ; 60 est un ART typique.
Graine	42	Même graine, même structure et même date de référence donnent le même espace de données — idéal quand tous les participants doivent voir la même chose.
Dernier état	aujourd'hui	Date du dernier état ; les autres se situent 14 jours avant, chacun. Le feu de qualité du rapport de portefeuille classe ainsi les données comme actuelles.

Réglage	Défaut	Signification
Langue des textes	langue de l'interface	Langue de l'histoire, du README et des briefings delta ; suit l'interface.

Détails pour analystes. Cycle times, motifs de flux et longueur de PI peuvent être surchargés par ART — les mêmes réglages que pour l'ART seul. Dans cette bêta, cela passe par la ligne de commande avec `--art-profiles FICHIER.json` (chapitre 6) ; le tableau dans l'interface suivra avec l'onglet ART.

Sources livrées avec l'espace. Toujours incluses : exports Jira par ART (sans personnes), réponses SLO Prometheus, déploiements/PR/incidents GitHub pour la première solution, valeurs DORA quotidiennes GitLab pour la seconde, mesures de qualité SonarQube et un CSV de SLO pour la voie manuelle — données brutes au format de chaque système avec les mêmes chiffres que les registres, de sorte que `sources fetch` tourne dans l'espace de données sans réseau.

Ébauches IA (interrupteur, désactivé par défaut). Un exposé de situation factice, trois réponses, un lot de prompts et une réponse d'exercice aux chiffres faux arrivent dans ai_drafts/ — tous avec le marqueur IA, aucun approuvé. Vous y exercez l'approbation et `llm check` sans installer de modèle.

La ligne **Estimation** à la fin indique exports Jira, registres, états et deltas pour vos réglages ; le pied de page résume modèle, périmètre, états, réalisme et graine.

3.3 Étape 3 — Générer

Dossier cible. Il est créé. S'il n'est pas vide, le générateur demande — « Générer quand même » écrase les fichiers de même nom, « Choisir un autre dossier » revient en arrière. « Parcourir … » ouvre le sélecteur de dossier ; là où il n'y en a pas, saisissez le chemin.

Progression. « Générer l'espace de données » lance neuf étapes ; chacune affiche en cours, terminé ou sauté, avec un détail. La durée va de quelques secondes à quelques minutes selon le périmètre (situation standard environ 10 secondes).

Étape	Ce qui est produit
Structure et histoire	structure.json et story.json : rôles liés, événements fixés
Exports Jira et classeurs	par ART un export Jira sous raw/ et IssueTimes/Transitions/CFD sous solutions/…/arts/
Registres	par solution les registres (risques, dépendances, NFR, SLO, DORA, capacités, thèmes …)
Configurations	portfolio.json, solution.json par solution, pi_config.json
Données brutes des fournisseurs	sources/<solution>/ avec fichiers de relecture et configurations sources
README	README.md avec structure, histoire, états et autres voies
Série d'états	snapshots/snapshot_<nn>_<date>.json par état, stands.json, prev/now
Briefings delta	deltas/delta_<nn>-<nn+1>.md et .html par paire consécutive

Étape	Ce qui est produit
Ébauches IA (factices)	ai_drafts/ — seulement avec l'interrupteur, sinon « sauté »

Annuler s'arrête après l'étape en cours ; le dossier contient ce qui a été construit jusque-là. Une erreur apparaît avec son message dans la liste.

Carte de résultat. Après la dernière étape apparaît « Espace de données prêt » avec durée et graine, quatre chiffres (tickets et ARTs, registres et solutions, états, types de sources), l'histoire en trois phrases et des boutons : Ouvrir le rapport de portefeuille, Dossier de conférence, Dernier delta, README, Ouvrir le dossier, Ébauches IA (si générées). Les rapports sont rendus au premier clic et ouverts dans le navigateur.

Comparer un delta. Choisir deux états (par exemple 3 → 4) et « Ouvrir le briefing delta » — toutes les paires consécutives sont prêtes sous deltas/.

Ensuite. « Nouvel espace de données avec un autre modèle … » revient à l'étape 1 ; la ligne en dessous montre la commande de poursuite (chapitre 7).

4 Ce que contient le dossier

Le dossier cible est un **espace de données de portefeuille** : tous les chemins des configurations sont relatifs à leur fichier de configuration. Le dossier peut être déplacé, copié ou compressé d'un bloc.

Chemin	Contenu
`structure.json`	Structure en données : portefeuilles → solutions → ARTs, workflow par solution
`story.json`	Histoire en données : modèle, rôles liés, événements avec interrupteur et fenêtre d'états, notes
`README.md`	Description de la structure, de l'histoire, des états, des sources et des autres voies — dans la langue de l'exécution
`portfolio.json`	Point d'entrée : configuration de portefeuille sur toutes les solutions
`pi_config.json`	Intervalles de PI sur la période des données
`solutions/<name>/`	par solution solution.json (stage_map propre si choisie), arts/ avec IssueTimes, Transitions et CFD par ART, registers/ avec les registres sous leurs noms standard
`raw/`	Exports Jira par ART au format REST API — en même temps matière démo pour get_data
`sources/<name>/`	Données brutes des fournisseurs en fichiers de relecture (Prometheus, GitHub ou GitLab, SonarQube, CSV) plus configurations sources
`snapshots/`	snapshot_<nn>_<date>.json par état, index stands.json, snapshot_prev/now.json = les deux derniers
`deltas/`	delta_<nn>-<nn+1>.md et .html par paire consécutive — déjà rendus
`ai_drafts/`	seulement avec l'interrupteur : exposé factice, réponses, lot de prompts, réponse d'exercice — tous avec le marqueur IA

Le dossier est directement utilisable — rapport de portefeuille, briefing delta de n'importe quelle paire, dossier de conférence, `sources fetch` sans réseau sur les fichiers de relecture (dans sources/<solution>/) et le contrôle d'export de get_data :

```
python -m portfolio portfolio.json
python -m portfolio --delta snapshots/snapshot_prev.json snapshots/snapshot_now.json
python -m portfolio portfolio.json --conference mappe.html
python -m sources fetch
python -m get_data check raw/<art>_jira.json
```

Avec plusieurs portefeuilles (modèle « Grande » ou structure propre), chaque portefeuille devient son propre espace de données dans un sous-dossier ; un README dans le dossier principal y renvoie. Une image de situation sur plusieurs portefeuilles n'est pas une fonction aujourd'hui — les données pour cela sont déjà côte à côte.

5 Comprendre l'histoire

Chaque modèle raconte la même situation depuis tous les registres : risques, dépendances, hypothèses, SLO et alarmes pointent vers les mêmes endroits. Les événements se lient à des **rôles** qui découlent de la structure — l'histoire reste ainsi cohérente avec votre propre structure.

Les rôles :
- **Première solution** (« hôte ») : paraît saine, porte les motifs silencieux ; son **dernier ART est la valeur aberrante** ; entre ses ARTs se trouvent la dépendance bloquée et la couture récurrente.
- **Seconde solution :** son **dernier ART est la source de données faible** ; c'est ici que l'« Order Sync API » rompt son SLO et qu'apparaît le nouveau risque de sécurité.
- **Couture inter-solutions :** le premier ART de la seconde solution dépend du premier ART de la première — visible seulement dans le rapport de portefeuille.
- Les autres solutions et portefeuilles reçoivent le jeu de base généré par la graine, sans ancres d'histoire.

Ce que la structure ne fournit pas tombe avec une **note** (à l'étape 1 et dans story.json) : avec une seule solution, il n'y a ni source faible, ni rupture de SLO, ni couture inter-solutions ; si la première solution n'a qu'un ART, la valeur aberrante, la dépendance bloquée et la couture récurrente manquent. Les dix événements :

N°	Ce qui se passe	Visible comme
E1	L'ART aberrant a environ trois fois le cycle time.	Rapport de comparaison : médiane/P95 en rouge ; alarme « P85 au-dessus de 30 jours » déclenchée ; image de livraison rare et lente ; une capacité at risk.
E2	La source faible livre des données faibles : pas de CFD, presque pas de first dates, données vieilles de 60 jours.	Confiance medium ; un risque vieillit (50 jours) ; une capacité critique ; un problème de flux survit à quatre conférences ; une hypothèse dépasse sa date de revue ; DORA/qualité low.
E3	La dépendance entre deux ARTs de la première solution est bloquée et en retard.	Dépendance at_risk → blocked ; heatmap en rouge ; alarme « deux bloquées » juste sous le seuil.
E4	La couture inter-solutions passe sous pression ; les value streams tirent l'un sur l'autre.	Plusieurs dépendances glissent de on_track à blocked en passant par at_risk — visible seulement dans le rapport de portefeuille ; base de l'indicateur de points de décision.
E5	La même couture ouvre pour la troisième fois une solution area sur le même sujet (environnement de test).	Deux solution areas fermées, une nouvelle — le signal EA : vérifier le découpage ou l'architecture, pas les équipes.
E6	L'« Order Sync API » de la seconde solution rompt son SLO.	Registre SLO : breached ; NFR « API response time » violé ; un écart de runway en retard.

N°	Ce qui se passe	Visible comme
E7	La première solution paraît saine — avec les motifs silencieux.	Un risque owned depuis 45 jours (aging) ; un thème sans epic (« declared & forgotten ») ; un problème de flux survit à trois conférences ; une capacité sans ART ; SLO « Checkout » at risk.
E8	La qualité des données de la source faible se dégrade encore.	La part de tickets sans first date monte à 80 % — confiance medium → low, couverture visible dans le titre.
E9	Nouveau risque dans la seconde solution : security review non planifiée.	Un risque élevé, atténué, apparaît — nouveau dans le briefing delta.
E10	Feuilles de route mises à jour.	Un epic perd son thème (initiative zombie) ; un epic de la seconde solution est tiré de P2 vers P1.

Fenêtres d'états. Chaque modèle fixe dans quels états un événement s'applique. Situation standard : valeur aberrante, source faible, rupture de SLO et motifs silencieux dans tous les états, les retournements (blocage, coutures, dégradation, nouveau risque, feuilles de route) seulement dans le dernier. L'Escalade les échelonne sur la série, le Rétablissement les inverse : l'état 1 montre tout, puis les choses guérissent l'une après l'autre. Le README de l'espace de données liste la fenêtre de chaque événement ; avant et après, la série montre la situation sans lui.

6 L'espace de données en ligne de commande

Tout ce que contiennent les trois étapes existe aussi comme option de `python -m testdata_generator --scenario portfolio`. Avant de construire, la ligne de commande affiche structure, interrupteurs et estimation ; après, dossier, états et durée.

Option	Défaut	Signification
--scenario portfolio	—	Générer un espace de données au lieu d'un ART seul
--output DOSSIER	(obligatoire)	Dossier cible (il est créé)
--structure small|standard|large|FILE.json	standard	Modèle de structure ou structure.json propre (portefeuilles → solutions → ARTs, workflow par solution)
--arts N[,N...]	—	Un portefeuille à partir du nombre d'ARTs par solution, p. ex. 3,4s,1 — trois solutions avec 3, 4 et 1 ARTs ; un s final donne à la solution une stage map propre. Non combinable avec --structure.
--story calm|standard|escalation|recovery	standard	Modèle d'histoire (calm, standard, escalation, recovery)
--event ID=on|off	—	Activer ou désactiver un événement, p. ex. --event E3=off --event E9=on ; répétable. Un événement dont le prérequis est désactivé reste désactivé, avec une note.
--stands N	6 (calm 3)	Nombre d'états (2–12), espacés de 14 jours, le dernier à la date de référence ; chaque paire consécutive reçoit un briefing delta sous deltas/
--scale s|m|l	m	Volume des registres par solution : s = ancres seulement, m = démo réaliste, l = test de charge
--seed N	(aléatoire)	Graine pour une sortie reproductible ; l'interface prend 42
--art-profiles FILE.json	—	Réglages par ART en JSON, p. ex. {"Alpha-1": {"mean_cycle_days": 30, "pattern": "cluster", "pattern_strength": 80}}. Champs : issue_count, mean_cycle_days, std_cycle_days, completion_rate, todo_rate, backflow_prob, pattern, pattern_strength, pi_duration_weeks, realism. Les surcharges valent dans tous les états ; le reste garde sa valeur d'histoire.
--ai-drafts	désactivé	Écrire des ébauches IA factices avec marqueur dans ai_drafts/
--lang de|en|ro|pt|fr	langue de la suite, sinon en	Langue de l'interface, de la console et des textes générés

Option	Défaut	Signification
--continue DOSSIER --to-date DATE	—	Poursuivre un espace de données existant jusqu'à la date (chapitre 7)
--web / --port N / --no-browser	—	Lancer l'interface navigateur : serveur local sur 127.0.0.1 (port 0 = libre aléatoire), clé d'accès dans le lien ; --no-browser n'affiche que le lien. Fin par Ctrl+C.
--dataspace	—	Lancer la fenêtre de l'espace de données — les mêmes trois étapes sans serveur

Exemples

```
# Situation standard, comme dans le tutoriel et les manuels
python -m testdata_generator --scenario portfolio --output demo/ --seed 42

# Escalade sur huit états, test de charge, textes en allemand
python -m testdata_generator --scenario portfolio --output demo-esc/ \
    --story escalation --stands 8 --scale l --lang de

# Structure propre, un événement désactivé, exercice IA activé
python -m testdata_generator --scenario portfolio --output demo-3/ \
    --arts 3,4s,1 --event E3=off --ai-drafts

# Cinq portefeuilles, 49 ARTs — nettement plus long
python -m testdata_generator --scenario portfolio --structure large \
    --output demo-large/
```

Sans --lang et sans réglage du lanceur, console et textes générés sont en anglais. Pour le français, passez --lang fr ou changez le lanceur.

7 Poursuivre un espace de données

Un espace de données peut être poursuivi jusqu'à une date ultérieure — comme un second extrait de Jira : le passé reste, les tickets ouverts avancent, de nouveaux s'ajoutent.

```
python -m testdata_generator --scenario portfolio \
    --continue demo/ --to-date 2027-01-15
```

Le générateur poursuit sur place les exports Jira, IssueTimes/Transitions/CFD, les configurations et la configuration de PI ; l'ancien snapshot_now devient snapshot_prev, le nouvel état snapshot_now, et le README reçoit une note avec l'ancienne et la nouvelle date de référence et le nombre de nouveaux tickets. Les registres restent inchangés. Les réglages viennent de l'enregistrement du générateur dans les fichiers ; la langue suit `--lang`.

La carte de résultat montre la commande adéquate sous « Ensuite ». Pour un seul export Jira, il existe --continue OLD.json (chapitre 8).

8 Un seul ART (générateur classique)

La voie classique produit **un** fichier d'export Jira au format REST API à partir d'un fichier de workflow : les tickets traversent les étapes, y séjournent des durées variables, reculent parfois et se ferment à un taux réglable. Le fichier entre directement dans transform_data et build_reports. Les réglages de cycle time et de motifs de flux sont ceux que l'espace de données surcharge par ART via --art-profiles.

8.1 La fenêtre

TestdataGenerator.bat (ou `python -m testdata_generator` sans option) ouvre la fenêtre avec des champs pour tous les paramètres. Seul le fichier de workflow est obligatoire ; le reste a des valeurs par défaut. La zone « Portefeuille démo » en bas construit la situation standard d'un clic — l'espace de données du chapitre 3 fait davantage.

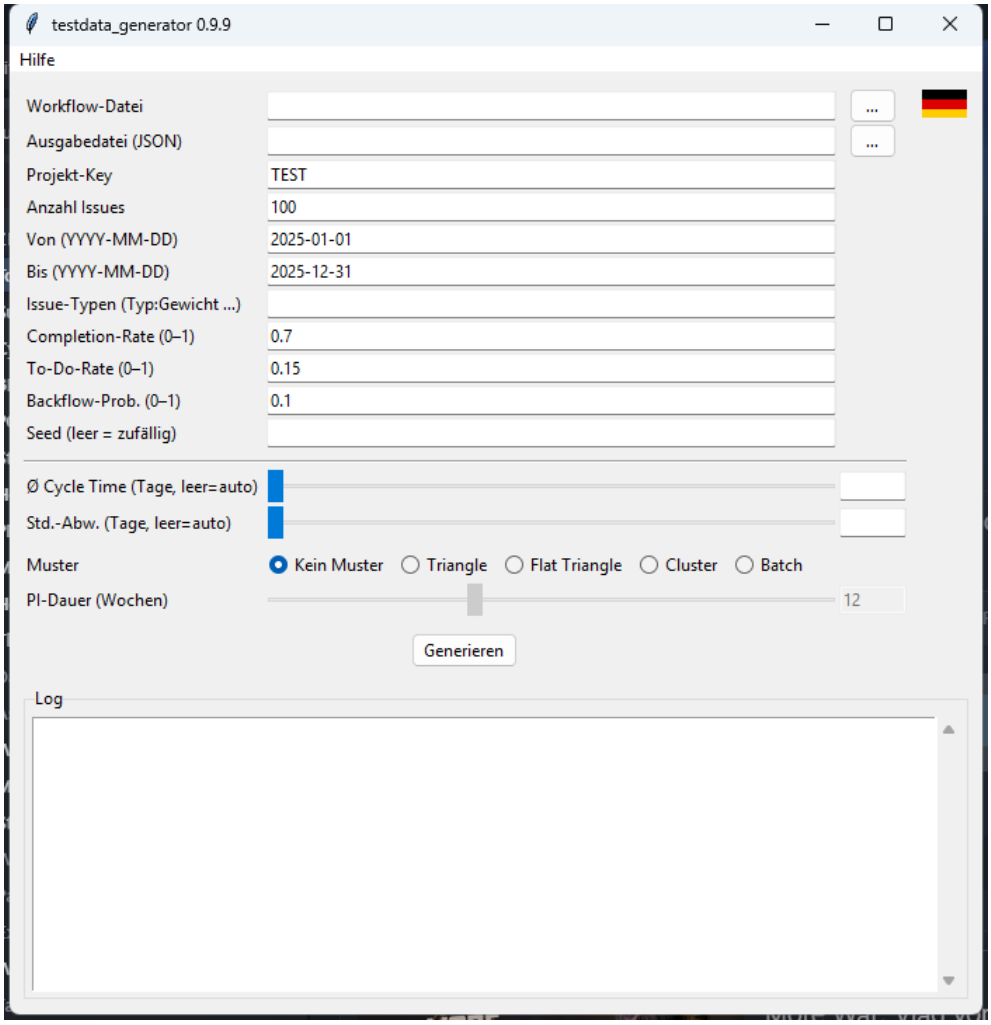


Fig. 2 : Fenêtre du générateur classique pour un seul ART

8.2 Le fichier de workflow

Le fichier de workflow définit les étapes de votre tableau Jira et est le seul champ obligatoire. Il a le même format que dans transform_data — un fichier sert aux deux modules. Chaque ligne décrit une étape ; les alias (noms de statut alternatifs dans l'export Jira) sont séparés par des deux-points. Deux lignes spéciales marquent le début et la fin du travail actif :

```
NomCanonique:Alias1:Alias2
<First>PremiereEtapeActive
<Closed>EtapeFermee
```

Ligne	Signification
Name:Alias1:Alias2	Étape avec un ou plusieurs noms de statut Jira alternatifs
Nom (sans :)	Étape sans alias
<First>Name	Première étape active — le cycle time commence ici
<Closed>Name	Étape fermée — marque un ticket comme terminé

L'exemple fourni `workflow_ART_A.txt` modélise un workflow SAFe ART typique :

```
Funnel:New:Open:To Do
Analysis:In Analysis:Estimated
Program Backlog:Ready for Dev:Backlog
Implementation:In Implementation:In Review:In Progress
Blocker
Validating on Staging:QA
Deploying to Production:Ready for Development
Releasing:Completed
Done:Canceled
<First>Analysis
<Closed>Releasing
```

« Funnel » a quatre alias — les quatre statuts Jira se normalisent en Funnel. Analysis est la première étape active (le cycle time commence ici). Releasing compte comme fermé ; Done (après Releasing) accueille aussi les tickets annulés.

8.3 Référence des paramètres

Tous les paramètres existent dans la fenêtre et en ligne de commande.

Paramètre	Défaut	Description
--workflow FILE	(obligatoire)	Fichier de définition du workflow (.txt)
--output FILE.json	_generated.json	Chemin du fichier de sortie
--project KEY	TEST	Clé du projet Jira des tickets générés
--issues N	100	Nombre de tickets
--from-date YYYY-MM-DD	2025-01-01	Date de création la plus ancienne
--to-date YYYY-MM-DD	2025-12-31	Date de transition la plus tardive
--issue-types TYPE:W ...	Feature:0.6 Bug:0.3 Enabler:0.1	Types de tickets avec pondérations (la somme n'a pas à faire 1)
--completion-rate FLOAT	0.7	Part des tickets atteignant l'étape fermée (0–1)
--todo-rate FLOAT	0.15	Part des tickets ouverts restant dans les premières étapes (0–1)
--backflow-prob FLOAT	0.1	Probabilité d'un pas en arrière par transition (0–1)
--realism 0-100	0	Désordre des parcours de statut : sauts, abandons, mises en attente, boucles, réouvertures ; environ 70 comme un vrai ART
--seed INT	(aléatoire)	Graine pour une sortie reproductible
--mean-cycle-days FLOAT	—	Cycle time moyen en jours (log-normal) ; active le pilotage du cycle time
--std-cycle-days FLOAT	30 % de la moyenne	Écart-type du cycle time en jours
--pattern NAME	none	Motif de flux : none / triangle / flat_triangle / cluster / batch (voir 8.4)
--pi-duration-weeks INT	12	Longueur du PI en semaines (pour cluster et batch)
--pattern-strength 0-100	50	Intensité du motif (0 = subtil, 50 = défaut, 100 = fort)
--continue OLD.json	—	Poursuivre un extrait antérieur jusqu'à --to-date : le passé reste octet pour octet, les tickets ouverts avancent, de nouveaux s'ajoutent ; nécessite --workflow
--learn-from FILE.xlsx	—	Apprendre un profil de transition depuis un vrai fichier de transitions (comptages et heures seulement) ; --transition-profile génère avec

8.4 Motifs de flux

Le générateur simule des anti-motifs de flux typiques issus des travaux de Vacanti et Singh. Combinez --mean-cycle-days avec --pattern ; la fenêtre offre des boutons radio et des curseurs.

Motif	Nuage de points	Description
none	Nuage de points régulier	Cycle time aléatoire sans tendance — comportement par défaut
triangle	Triangle, angle droit en bas à droite	Le cycle time monte linéairement dans le temps (environ 1–4× à l'intensité par défaut). Un processus qui ralentit de plus en plus.
flat_triangle	Triangle, la montée s'aplatit	Comme triangle, la montée s'aplatit vers la fin. Une dégradation qui se stabilise.
cluster	Amas verticaux en fin de PI	Le cycle time reste stable, mais la plupart des tickets se ferment dans les deux dernières semaines avant la fin du PI. Comportement dicté par l'échéance.
batch	Colonnes de hauteur variable en fin de PI	Comme cluster, mais avec un cycle time très variable (0,1× à 3× la moyenne). Tâches courtes et très longues sont poussées ensemble en fin de PI.

--pattern exige --mean-cycle-days ; sans lui le motif est ignoré. --std-cycle-days règle la dispersion, --pi-duration-weeks la longueur du PI pour cluster et batch.

8.5 ART_A — exemple complet

Du générateur au rapport fini sur le projet fictif ART_A :

Étape 1 : Générer les données de test

```
python -m testdata_generator \
    --workflow testdata_generator/workflow_ART_A.txt \
    --project ART_A --issues 200 \
    --from-date 2025-01-01 --to-date 2025-12-31 \
    --issue-types Feature:0.6 Bug:0.2 Enabler:0.2 \
    --completion-rate 0.75 --backflow-prob 0.08 \
    --seed 42 --output ART_A_generated.json
```

Étape 2 : Traiter avec transform_data

```
python -m transform_data ART_A_generated.json \
    --workflow testdata_generator/workflow_ART_A.txt
```

Étape 3 : Créer le rapport avec build_reports

```
python -m build_reports
```

À quoi s'attendre (graine 42, 200 tickets) :
- Clés ART_A-0001 à ART_A-0200
- Types environ 120 Feature, 40 Bug, 40 Enabler
- Environ 150 tickets en Releasing ou Done
- Dates de création réparties sur 2025
- Pas en arrière occasionnels (backflow-prob 0.08 = environ 8 %)

9 Questions fréquentes

Fenêtre ou navigateur — lequel prendre ?

La fenêtre. Elle produit le même espace de données et n'ouvre aucun port réseau. La page du navigateur seulement sur un ordinateur privé ; sur les ordinateurs d'entreprise, le logiciel de sécurité bloque et signale les serveurs locaux.

Pourquoi mon espace de données est-il en anglais alors que je parle français ?

Sans indication de langue, le générateur prend la langue de la suite du lanceur, sinon l'anglais. Réglez le lanceur sur le français, basculez en haut à droite ou passez --lang fr.

Le dossier cible n'est pas vide — que se passe-t-il ?

Le générateur demande. « Générer quand même » écrase les fichiers de même nom et laisse le reste ; « Choisir un autre dossier » abandonne.

La génération prend du temps.

À partir de 30 ARTs, ou au périmètre L avec plusieurs portefeuilles, l'encadré « Ce qui est produit » avertit. Pour la formation, la structure standard et le périmètre M suffisent (environ 10 secondes).

Données de test ou vraies données ?

Pour les démos, la formation et les tests d'installation, les données de test sont le meilleur choix — sans question de confidentialité. Les vraies analyses de flux et les décisions demandent de vraies données ; l'export manuel est décrit dans le manuel Get Data.

Puis-je régler certains ARTs différemment ?

Oui, via --art-profiles FICHIER.json en ligne de commande (chapitre 6). L'onglet correspondant dans l'interface suivra dans une bêta ultérieure.

10 Glossaire

Terme	Explication
ART	Agile Release Train — plusieurs équipes qui livrent ensemble dans SAFe
Espace de données	Un dossier avec tout ce dont une image de situation de portefeuille a besoin : configurations, exports, registres, états, briefings — autonome et déplaçable
Briefing delta	« Qu'est-ce qui a changé ? » entre deux états — débit, confiance, coutures, nouveaux risques
Dossier de conférence	Le dossier pour la value stream conference : problèmes de flux qui survivent aux conférences, avec l'image de situation
Couture	Une dépendance entre ARTs ou solutions où la pression se montre — au sein d'une solution ou entre value streams
Registre	Une liste par solution : risques, dépendances, hypothèses, NFR, runway, SLO, DORA, capacités, thèmes, problèmes de flux …
État	Un instantané figé de l'image de situation à une date ; les états sont espacés de 14 jours
Graine	Valeur de départ du générateur aléatoire — même graine, même sortie
Solution	Un groupe d'ARTs qui livrent ensemble une solution ; dans l'espace de données, le niveau avec ses propres registres
Stage map	Regroupement propre des étapes de workflow d'une solution (p. ex. amont/réalisation/terminé) au lieu du chemin par défaut
Value stream	Le flux du besoin à la valeur ; représenté dans l'espace de données par les solutions
Workflow	Suite ordonnée d'étapes qu'un ticket traverse ; décrite dans un fichier texte
Réalisme	Degré de désordre des parcours de statut : sauts, abandons, mises en attente, boucles, réouvertures
CFD	Cumulative Flow Diagram — stock par étape au fil du temps, cumulé